

(PCC) Data Structures and Algorithms

Teaching Scheme

Lectures: 3 Hrs/ Week

Laboratory: 2 Hrs/ Week

Self-Study: 1 Hr/Week

Evaluation Scheme

Theory: MSE: 30 Marks, TA: 20 Marks, ESE: 50 Marks

Laboratory: CIE: 100 Marks

Course outcomes

Students will be able to:

1. Understand the fundamental concepts of various data structures (e.g., arrays, linked lists, stacks, queues, trees, graphs, hash tables).
2. Understand the concepts of linear and non-linear data structure and their representation
3. Demonstrate the ability to write abstract data types and the use of basic data structures in solving problems.
4. Discuss, compare and implement algorithms for various operations in different implementations of stacks and queues, tree, graph, matrices, heap, etc.
5. Analyze the time complexity of different searching, sorting, and traversal algorithms.

Course content

Fundamental Concepts

[6 Hrs]

Introduction to Data Structures: Data, Data Object, Data types, Abstract Data Types (ADT), Data structures, Concept of primitive and non-primitive, linear and non-linear data structures. Introduction to Algorithms: Definition and Characteristics of an algorithm, Algorithm Specification, Performance Analysis- Time and space complexity, Asymptotic notations, Best, Average and worst cases. Introduction to Data representation and files: Text and binary files, use of various libraries for handling files.

Linear Data Structures and their storage representation

[6 Hrs]

Features of Linear Data Structures, Array as ADT, List as ADT, Concept of linked linear list, Operations on linked linear list, Singly linked list, doubly linked list, circular linked list. Application of linked linear list- Polynomial manipulations, linked dictionary.

Linear Data Structures: Stacks and Queues

[8 Hrs]

Stack and queue as ADT, Operations on stack and queue. Implementations using arrays and linked lists. Application of stack for expression conversion and evaluation, Recursion. Problems like maze and knight's tours

Non-Linear Data Structures: Trees

[6 Hrs]

Concept of Non-Linear Data Structures, Tree: Basic terminology, representation of trees, Binary Tree: ADT Binary trees, Properties of a Binary Tree and its representations, Binary tree traversals – Inorder, preorder, postorder (recursive and non-recursive) and various operations. Binary Search Tree (BST): Definition, searching a BST, Operations of Insertion and deletion on BST. Heaps: Priority queues, Max and Min Heap, Operations on heap

Non-Linear Data Structures: Graphs

[8 Hrs]

Graph ADT, Representation of graphs using adjacency matrix, adjacency list, Elementary graph operations- Breadth First Search (BFS), Depth First Search (DFS), Analysis of BFS and DFS Spanning Trees- Introduction, obtaining minimum cost spanning tree using greedy design strategy of Prim's and Kruskal's algorithms, Single source shortest paths using Dijkstra's algorithm.

Searching and Sorting

[8 Hrs]

Linear and binary search algorithms and their analysis. Bubble Sort, Selection sort, Insertion Sort, Quick Sort, Heapsort with time complexity analysis. Hashing: hashing functions, chaining, overflow handling with and without chaining, open addressing: linear and quadratic probing.

Self-study

[12 Hrs]

Content

Textbooks

1. E. Horowitz, S. Sahni, S. Anderson-freed, "Fundamentals of Data Structures in C", Second Edition, University Press, ISBN 978-81-7371-605-8
2. B. Kernighan, D. Ritchie, "The C Programming Language", Prentice Hall of India, Second Edition, ISBN 81-203-0596-5
3. Y. Langsam, M. Augenstein and A. Tannenbaum, "Data Structures using C", Pearson Education Asia, First Edition, 2002, ISBN 978-81-317-0229-1

Reference Books

1. Ellis Horowitz, S. Sahni, D. Mehta "Fundamentals of Data Structures in C++", Galgotia Book Source, New Delhi 1995 ISBN 16782928
2. Jean-Paul Tremblay, Paul. G. Sorensan, "An introduction to data structures with Applications", Tata Mc-Graw Hill International Editions, 2nd edition 1984, ISBN-0-07- 462471-7

Web references

1. <https://nptel.ac.in/courses/106102064>
2. <https://nptel.ac.in/courses/106106127>
3. <https://nptel.ac.in/courses/106105171>

Laboratory assignments

Suggested List of Assignments:

1. Write a program to remove duplicate doubles from an array of doubles. In the program, write a function which accepts an array of doubles and removes the duplicates from the array and has return type void.
2. Compare the time complexity of two sorting algorithms, by following the given steps. Create a set of data files with count of integers varying into thousands and millions. Sort the files using both the algorithms. Plot graph of the time taken by both the programs using tools like gnuplot. Compare the graphs and comment on the time complexity theoretically predicted and practically observed.
3. Write a function which evaluates an infix expression, without converting it to postfix. The input string can have spaces, (,) and precedence of operators should be handled.
4. Implement a queue (that is write queue.c and queue.h only) of characters, such that on an enqueue, the char is added at the end of queue, and on a dequeue the first element is taken out, but the queue uses only a 'head' pointer and not a 'tail pointer.
5. Write a data type called "Integer". The data type should represent integers of unlimited length.
6. Write a sorting program with the following features: Reads data from a text file and sorts it

alphabetically by default. If the file has data in rows and columns (separated by space or tab) then allows sorting on a particular column. Allows any sort using numeric or alphabetical ordering.

7. Write the following functions for a binary search tree implementation: Searches the maximum value in the tree, preorder traversal without using recursion, Search the string in the tree and returns a pointer to the node, print the binary tree so that it looks like a tree.
8. Write code to list leaf nodes, non-leaf nodes and level of all nodes in a given binary tree.
9. Write a code for level order traversal of a binary tree with and without stack.
10. Start with an empty AVL tree and perform a series of insertions like : December, January, April, March, July, August, October, February, November, May, June. Display the tree.
11. Implement a sparse matrix with operations like initialize an empty sparse matrix, insert an element, sort a sparse matrix on a row-column, add two matrices and return the result as a matrix, transpose a matrix, etc.
12. Develop C functions to insert and delete into/from a max heap under the assumption that a dynamically allocated array is used, the initial capacity of this array is 1, and array doubling is done whenever we are to insert into a max heap that is full.
13. Implement Heap sort algorithm on a set of records, with a specified key.
14. Write a graph implementation, using adjacency lists and demonstrate Dijkstra's algorithm on it.
15. Write a program to read a map stored in a text file with (city1, city2, distance) comma separated values. Build a graph using this data. Print all pairs shortest paths information between all pairs of cities.
16. Implement DFS and BFS on a Graph.
17. Write a program to find all connected components of a Graph, on a map specified in a text file as Source, Destination, distance comma separated values.
18. Develop a hash table implementation in which overflows are resolved using chaining. Read a set of records from a file, insert them into a hash table, then perform a set of searches using supplied data and show the search results.
19. Implement a dictionary using a sparse matrix data structure.

eMini-project: Write an application of your own demonstrating your skills in defining a problem, writing down the requirements carefully, designing a modular solution with clear separation of abstract data types and their use, design of proper function prototypes and division of work among functions. The application can be a unix command re-implemented (e.g. cut, find, tar, fdupes, bc, etc.), reimplementation of C library functions, memory allocator, a simple game using libraries like n-curses or SDL, games like sudoku or chess, or an application to manage institutions like hospitals, colleges, shops, etc